MACHINE LEARNING LABORATORY - OBSERVATION

1. Experiment No.: 5	**Date:** 29/10/26

2. Experiment Question

Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python.

3. Aim

To implement the K-Nearest Neighbours classification algorithm in Python, to study the effect of K on accuracy, and to classify a new sample of the Iris data set.

4. Concept

K-NN is a simple, instance-based (lazy) learning algorithm. It does not build a model during training; it just stores the training samples. To classify a new sample it finds the K training samples closest to it (usually by Euclidean distance) and assigns the class that is most common among those K neighbours.

- A small K gives flexible boundaries but is sensitive to noise (over-fitting).
- A large K gives smoother boundaries but may blur class differences (under-fitting).
- An odd K avoids ties in a two-class problem. Features of very different ranges should be scaled before applying K-NN.

5. Dataset Description

The Iris data set (Fisher, 1936) has 150 samples of iris flowers with 4 numeric features and 3 balanced classes. It is split 70:30 into 105 training and 45 testing samples (stratified, random_state = 42).

Item	Details	Notes
Features	Sepal length, sepal width, petal length, petal width	All in centimetres
Classes	Setosa, Versicolor, Virginica	50 samples each
Training / testing	105 / 45	Stratified split

6. Algorithm

1. Store all training samples with their class labels.
2. Choose the value of K.
3. For a query sample, compute its distance to every training sample.
4. Sort the distances and select the K nearest training samples.
5. Count the class labels of these K neighbours.
6. Assign the majority class to the query sample.

7. Step-by-step Procedure

1. Load the Iris data set and split it into training and testing sets.
2. Create the KNN class with fit() (stores the data) and predict() (distance, sorting and voting).
3. Test K = 1, 3, 5, 7, 9 and 11 and record the accuracy for each.
4. Select the best K (K = 5) and display some test predictions.
5. Classify a new sample [6.1, 2.9, 4.8, 1.6] and list its 5 nearest neighbours.
6. Plot the training data with the new sample and the accuracy against K.

8. Formulae

Euclidean distance between a query x and a training sample y

$$d(x, y) = sqrt(SUM[(x_i - y_i)\^2]), \quad i = 1 \ldots n$$

Majority vote

$$class(x) = argmax \text{ over } c \text{ of } SUM[1(y_i = c)], \quad i \text{ in the } K \text{ nearest neighbours}$$

Accuracy

$$Accuracy = (number\ of\ correct\ predictions / total\ predictions) \times 100$$

9. Python Program

```
"""
Experiment 5: K-Nearest Neighbours (K-NN) classifier
Dataset : Iris (150 samples, 4 features, 3 classes)
"""
import numpy as np
from collections import Counter
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split


class KNN:
    def __init__(self, k=3):
        self.k = k


    def fit(self, X, y):
        self.X_train, self.y_train = X, y


    def _predict_one(self, x):
        # Euclidean distance to every training sample
        dist = np.sqrt(((self.X_train - x) ** 2).sum(axis=1))
        nearest = np.argsort(dist)[: self.k]              # indices of k nearest
        votes = Counter(self.y_train[nearest])            # majority vote
        return votes.most_common(1)[0][0]


    def predict(self, X):
        return np.array([self._predict_one(x) for x in X])


iris = load_iris()
X, y = iris.data, iris.target
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)


print("Training samples:", len(X_tr), "  Testing samples:", len(X_te))
print("\nEffect of K on accuracy:")
print(f"{'K':>3} {'Accuracy (%)':>14}")
acc_by_k = {}
for k in [1, 3, 5, 7, 9, 11]:
    model = KNN(k); model.fit(X_tr, y_tr)
    acc = np.mean(model.predict(X_te) == y_te) * 100
    acc_by_k[k] = acc
    print(f"{k:>3} {acc:>14.2f}")


K = 5
model = KNN(K); model.fit(X_tr, y_tr)
pred = model.predict(X_te)
print(f"\nChosen K = {K}   Test accuracy = {np.mean(pred == y_te)*100:.2f}%")
print("\nFirst 8 test predictions:")
print(f"{'Actual':<12}{'Predicted':<12}")
for a, p in list(zip(y_te, pred))[:8]:
    print(f"{iris.target_names[a]:<12}{iris.target_names[p]:<12}")


# classify a brand-new sample
new = np.array([[6.1, 2.9, 4.8, 1.6]])
dist = np.sqrt(((X_tr - new) ** 2).sum(axis=1))
idx = np.argsort(dist)[:K]
print("\nNew sample [SL, SW, PL, PW] =", new[0].tolist())
print("Its", K, "nearest neighbours:")
for i in idx:
    print(f"   distance = {dist[i]:.3f}   class = {iris.target_names[y_tr[i]]}")
print("Predicted class:", iris.target_names[model.predict(new)[0]])


# ------------------------------------------------------------------ plots
fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
marks = ["o", "s", "^"]; fills = ["white", "#999999", "black"]
for c in range(3):
    m = y_tr == c
    ax[0].scatter(X_tr[m, 2], X_tr[m, 3], marker=marks[c], c=fills[c], edgecolors="black",
                  s=28, label=iris.target_names[c])
ax[0].scatter(new[0, 2], new[0, 3], marker="*", s=220, c="white", edgecolors="black",
              linewidths=1.5, label="new sample")
ax[0].set_xlabel("Petal length (cm)"); ax[0].set_ylabel("Petal width (cm)")
ax[0].set_title("Training data and the new sample"); ax[0].legend(fontsize=8)
ax[1].plot(list(acc_by_k), list(acc_by_k.values()), "k-o")
ax[1].set_xlabel("K"); ax[1].set_ylabel("Accuracy (%)"); ax[1].set_title("Accuracy vs K")
ax[1].grid(True, color="#cccccc", lw=0.5)
plt.tight_layout(); plt.savefig("figs/exp5_knn.png", dpi=200); plt.close()
```

10. Output

```
Training samples: 105   Testing samples: 45


Effect of K on accuracy:
  K   Accuracy (%)
  1          93.33
  3          95.56
  5          97.78
  7          95.56
  9          95.56
 11          93.33


Chosen K = 5   Test accuracy = 97.78%


First 8 test predictions:
Actual      Predicted
virginica   virginica
versicolor  versicolor
virginica   virginica
versicolor  versicolor
virginica   virginica
virginica   virginica
versicolor  versicolor
versicolor  versicolor


New sample [SL, SW, PL, PW] = [6.1, 2.9, 4.8, 1.6]
Its 5 nearest neighbours:
   distance = 0.245   class = virginica
   distance = 0.300   class = versicolor
   distance = 0.332   class = versicolor
   distance = 0.361   class = virginica
   distance = 0.374   class = versicolor
Predicted class: versicolor
```

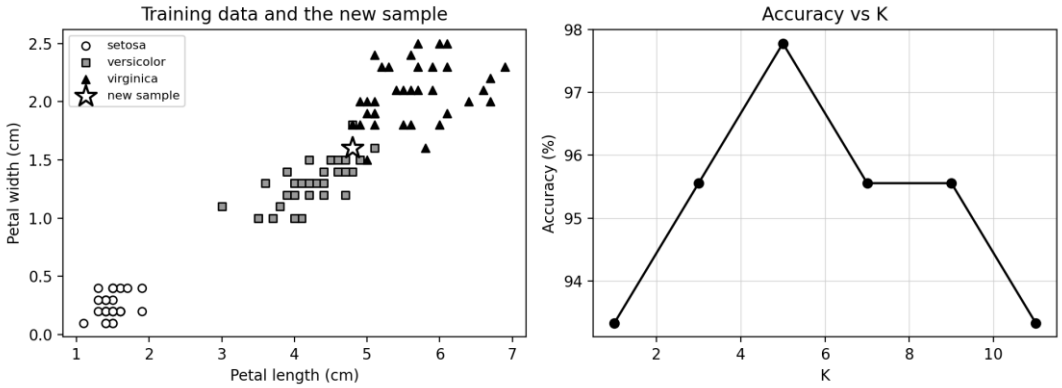


Fig. 5.1 Iris training data with the new sample (left) and accuracy for different K (right)

11. Result / Conclusion

The K-NN classifier achieved its best test accuracy of 97.78% at K = 5. Very small and very large values of K gave lower accuracy (93.33% at K = 1 and K = 11). The new sample [6.1, 2.9, 4.8, 1.6] has three Versicolor and two Virginica among its five nearest neighbours, so it was classified as Versicolor. Hence K-NN was implemented successfully in Python.